Full-Stack Group Lab 1 Documentation

Violet French, Jarrod Hutcheon, Brayden Hermanson, & Isaiah Guilliatt

October 29, 2026

Contents

1 Project Proposal **3**
- 1.1 Team Jobs . . . 3
  - 1.1.1 Violet . . . 3
  - 1.1.2 Isaiah . . . 3
  - 1.1.3 Brayden . . . 3
  - 1.1.4 Jarrod . . . 3

2 State Management - Violet **4**
- 2.1 Products . . . 4
- 2.2 Filtered-Products . . . 4
- 2.3 Search Filter . . . 4
- 2.4 Select Filter . . . 4
- 2.5 Product Selected . . . 4
- 2.6 Mode . . . 4

3 CRUD Psuedocode - Violet **5**

4 Developer Log **5**
- 4.1 Violet . . . 5
  - 4.1.1 Filtering . . . 5
  - 4.1.2 Product Grid . . . 6
  - 4.1.3 Socials Bar . . . 6
  - 4.1.4 CRUD Psuedocode . . . 7
  - 4.1.5 CRUD Implementation . . . 7
  - 4.1.6 Testing . . . 8
- 4.2 Brayden . . . 9
  - 4.2.1 Product Tile . . . 9
- 4.3 Jarrod . . . 9
  - 4.3.1 JSON File . . . 9
- 4.4 Isaiah . . . 9
  - 4.4.1 Interface . . . 9
  - 4.4.2 Header Component and Responsive Social Bar . . . 10

4.4.3 Footer Component and Dynamic Navigation 10
4.4.4 Global E-Commerce Theming and Palette 10
4.4.5 Form Styling, Container Spacing, and Product Grid Alignment 11

5 Testing **11**
5.1 Test Cases 12
5.1.1 Fetching Products 12
5.1.2 No Match Checks 13
5.1.3 No Filter Check 14
5.1.4 Shallow Sort Validation 14
5.1.5 CRUD 14
5.2 Debugging 15
5.2.1 Filtering - Violet 15
5.2.2 Sorting - Violet 16
5.2.3 Footer Viewport Floating - Isaiah 17
5.2.4 Header Social Bar Mobile Collision - Isaiah 18
5.2.5 Form Edge Collision and Focus Glow Feedback - Isaiah . . . 19

6 Figures **19**

List of Figures

1 Component Heirarchy Diagram 20
2 Interface Sketch 21

1 Project Proposal

The application we are creating is called GenStore. GenStore (short for generic store) is a mockup of an eCommerce site like Amazon or Etsy. GenStore is designed for any user whether they be someone looking to buy something or a company looking to sell their products to a mass market. The site manages information about products including but not limited to: product name, price, categories, seller, and many more. The site will display products and allow the user to filter or search for a desired product. The site will also allow users to update, delete, or create new product listings which will update in realtime.

1.1 Team Jobs

1.1.1 Violet

Violet was in charge of creating the filtering and searching functions. She also created much of the doucmentation including: this document, state Management Document, Component Heirarchy Diagram, and the README. Violet also served as the group leader delegating tasks among team members and herself and keeping people on track. Violet created the product grid which links together the filtering features with Brayden's product tiles. Violet wrote many test cases for the code and the function that runs those tests. Violet also created the social bars and tiles. Violet created the psuedocode and implementation for the CRUD operations.

1.1.2 Isaiah

Isaiah was in charge of web design and had the final say in how the page looked. Isaiah was primarily in charge of the CSS but was also responsible for the creation of static elements like the header and footer for the site. He created the interface sketch and built the site based off that plan.

1.1.3 Brayden

Brayden was in charge of creating the product tiles for displaying on the products grid. He had to format the data into a user-friendly format and structure the tiles to look nice.

1.1.4 Jarrod

Jarrod was in charge of the data for the application. Jarrod created the products in the JSON file and made sure they were pulled in properly. Jarrod structured the JSON file in a preplanned format so that other people knew what properties to expect.

2 State Management - Violet

2.1 Products

The products state is the central state of the program. Products is stored in App.jsx and contains all of the products for displaying.

2.2 Filtered-Products

The filteredProducts state stores the filtered products which is given out to the ProductTile components for displaying. Filtered products is changed when the filter inputs change or when the overall products change causing it to re-filter the products. The filtered products are what gets displayed to the user even if no filtering is currently active. Due to filteredProducts being used by the product tiles it is also stored in App.jsx where the product grid is at. Each of the filtered products gets passed as a prop to product tiles via a map.

2.3 Search Filter

Search filter stores the current text search query. When the input for the search input changes it causes the search filter to change which also causes a rerender of the product tiles to match the new query. The search filter contains the raw text of the search box and assessing the queries is done by the filterProducts function. Queries can use relational operators (>=, <=, >, <, =, ==) to check price or they can use '&' as a delimiter for multiple combined queries. This filter is stored in App.jsx.

2.4 Select Filter

The Select filter acts very similar to search filtering but provides a more accessible interface for filtering. Rather than using a text query to filter the user just clicks on an option in a dropdown box of categories for product. When the state of the dropdown changes the select filter does too triggering a rerender of the product tiles. This filter, like searchFilter, is stored in App.jsx.

2.5 Product Selected

The product selected state holds the ID of the product that is currently selected and is used for CRUD updates. This state is stored in App.jsx but the setter is passed down to the product tiles as they store information about each product tile. When a product tile is clicked this state is updated by the tile which is then used for the updating and deleting of products via the CRUD form.

2.6 Mode

Mode is a state hosted entirelly in App.jsx and keeps track of the current listing form mode they are in. The three modes are: create listing, delete listing, and update

listing. The mode state is only used for display purposes. The mode helps change the styling of the submit button, the text of the submit button, and the instructions displayed (instructions explain how that mode works). Mode gets updated when the dropdown for the listing form changes.

3 CRUD Psuedocode - Violet

```
PRODUCTS = STATE()
SELECTEDPRODUCT = STATE()

READ:
    FOR PRODUCT IN FILTERED-PRODUCTS:
        CREATE PRODUCT-TILE

ON PRODUCT TILE CLICK
    SELECTEDPRODUCT += PRODUCTID

CRUD MENU:
    DROPDOWN MENU - SELECT OPTION
        CREATE
        UPDATE
        DELETE
    PRODUCT INFO SECTION
        NAME
        CATEGORIES (COMMA DELIMITED)
        SELLER
        PRICE
        DESCRIPTION
        AVAILABLE - YES/NO
    SUBMIT BUTTON: ON-CLICK
        CREATE: PRODUCTS += NEW-PRODUCT     ⟺     ALL FIELDS ARE
POPULATED
        UPDATE: PRODUCT[ID] = UPDATED-PRODUCT
        DELETE: PRODUCTS -= PRODUCT[ID]  ⟺  PRODUCT IS SELECTED
        CLEAR SELECTED
```

4 Developer Log

4.1 Violet

4.1.1 Filtering

To implement the filtering I decided to copy the style I've used in the past few programs. I used both a category select option and a text search filter. When filtering the products it applies both of these to the list of products and returns the filtered list.

These filtered results gets given to the state setter for the filteredProducts state which is what is passed to the product grid for displaying, this way products and filteredProducts remain seperate. I also added alphabetical sorting like in my previous programs. I originally designed the filtering as its own component called FilterForm but realized near the end that this would not work very well because of how React state works. Instead I moved it up to App.jsx and redesigned the heirarchy. This wasn't a problem as I did this on my own time when the component heirarchy was still in its infancy. Going into this project we didn't have enough time to plan things out and the FilterForm is when I started to realize this. After setting everything up I cemented a component heirarchy diagram to guide further features now that we had a better idea of what our product will look like.

I wasn't happy that I had to move the logic back to App since I know it will make things very messy in the long run. With me moving it back to App it resulted in around 150 lines of code for something that would be far better as a component. There may be a way to do it as a component but it would be such a headache to set up that it would wind up making things more complex. This is definitely a pitfall of React. The increased complexity in writing dynamic code hurts how modular you can make the program. I do think with a database things would be a lot better since database langauges like SQL include filtering clauses and sorting clauses which would trivialize this. So in all honestly you can't blame React too much but I must say that state is a headache to deal with as compared to something like classes and properties you might be working with if you were using a basic templating engine such as Thymeleaf (less reactive but is also SSR)

4.1.2 Product Grid

The product grid underwent many forms. Initially it was slated to be a React component but I decided not to do that since I didn't see the point. This later changed when I realized it wasn't updating when filtered products updated. This motivated me to turn it back into a component so that when filteredProducts changed so to would the product grid. I also wanted to add a conditionally rendered span that would let you know if there were no matches. I initially placed this on the App level but this causes problems. Once the span triggers then it will never stop triggering. This is because the span isn't tied to any kind of state so once it triggers React will never need to rerender it meaning it will never attempt to mount the product grid even when you change the filters back. The solution is to move the span into the product grid component. With this, anytime the filteredProducts state variable changes it will rerender the ProductGrid. Anytime it renders the product grid it will check if there are any products matching and display them if there are or display the span if there aren't.

4.1.3 Socials Bar

I decided to add a bar for socials to add some more reusable components to the code. This created two new components: SocialTile and SocialBar. SocialBar is a wrapper element for all the SocailTiles and it will construct each tile inside itself. SocialTile takes in three parameters: graphic, alt, link. Graphic is a path to the image

that should be used for the link, this image will be forced to 20×20px so it won't be too big. Alt is the alternate text in case the image cannot load. Link is a link to whatever site or resource you want the image to link to. Image files should be stored in public/ since this is the folder for static files and will be served up to the user. Then the graphics path should be the path from public to the image file. Some tinkering was necessary to get things to work properly. It took me a little while to figure out how to get the image path right but eventually I realized I had mispelled the src attribute which is why it wouldn't display.

4.1.4 CRUD Psuedocode

I wrote the psuedocode for the CRUD directly in LaTeX. This was pretty challenging to get it to work right since I wanted things to look a specific way. I wound up placing the whole psuedocode inside a minipage environment so it could have monospaced font with inline math. Verbatim would prevent me from being able to add some inline math and no verbatim means monospaced is difficult. This is because I want the whole document to be in Times but the code to be in monospaced but LaTeX doesn't have native support for Times. Typically you import a package which makes everything monospaced but from what I understand there is no way to stop using Times for a moment and then switch back later. At least you can't do this for multiparagraph sections. I also wanted proper indentation for this which meant I had to write some very sloppy LaTeX to get it to look right. Overall though I am satisfied with how it came out.

I didn't face much challenge in writing the psuedocode. The READ was already complete when I started writing it so I just wrote some psuedocode of how it works. The other code just sort of played off what information we have for the products.

4.1.5 CRUD Implementation

Originally I was not in charge of the implementation but this changed later in development. I didn't really have any issues writing the CRUD as the psuedocode provided a perfect template. There was some difficulty in getting the checkbox to work properly since documentation on checkboxes in forms is sparse but eventaully I figured it out. I originally planned for the tiles to change colors when they were selected but due to the time crunch I decided to leave that out and instead add some other ways to indicate that a product was selected like the product name and an alert on clicking it. I also did a bit of styling just so stuff looked a little better but didn't do much for it.

I took on CRUD implementation because I thought Jarrod had done it all wrong when he did it so I wound up redoing it. After taking a closer look at his code I realized while there were issues with the code and some of the things I criticized him for were just incorrect. I have since apologized to him for my outburst and included some of his features in the code. One of his ideas was using a mode state to change the color and text of the submit button. Originally I didn't know why mode was being used for class and criticized him for this but eventually I realized what I had missed. I decided to re-add this feature. I made some changes so it no longer uses a detached function

instead just placing it right in the JSX as an arrow function to try to keep things a bit cleaner. Additionally I added his idea of having the different modes display a short description on how to use them. Instead of embedding it into the JSX I used a seperate function to keep things easy to read.

4.1.6 Testing

I was in charge of writing most of the testing code. I was not initially going to make robust test cases but I realized as I was testing that simple checks here and there were not sufficient and we needed a stream-lined was to do this. I did this by writing a test() function inside App.jsx that would test different parts of our code. Within this function there is a arrow function called fAssert() which is used to speed up assertion checks. It takes in three parameters: value, expected, msg. Value is the value you want to do the assertion check on and is the only required parameter. This is the value you want to compare to expected to see if it matches. Expected is the value you expect to get, if the value does not match expected there is an error that gets logged in the console. Expected has a default value of true meaning you can pass a conditional to value to quickly do an assertion check. The final parameter, msg, outputs a unique message when assertion fails. This msg will follow the text 'Assertion Fail: '.

The fAssert() function was originally called fassert since it would flushSync than assert. Flushing the sync is used to force a rerender meaning you will get the most recent state variables. Flushing sync was recommended by the React documentation for immediately getting an updated state. This did not work however since the code only runs on first render (mounting) so flushing the sync doesn't do anything. The solution was to instead directly set the state variables skipping past the state setting function. This is used in this case since we don't need a rerender we just need to update the value of the states that way it can be used internally for the assertion checks. Despite flushing being removed from the function I kept the name fAssert incidentally after doubling back on my code several times and just decided to keep the name. Below is the code for the fAssert() function.

```
const fAssert = (value, expected=true, msg="Assertion test failed") => {
    if (value !== expected) console.error("Assertion Fail: " + msg);
}
```

The test() function is called once on mount using an effect. This effect is only triggered if the variable TESTING_ENV is set to true otherwise testing will not occur. A few functions were redesigned during creation of test cases to allow them to be used on first mount.

Information about the different test cases is outlined in a later section.

4.2 Brayden

4.2.1 Product Tile

Overall, my side of the project wasn't too bad to work on, as UI is usually my favorite part of any project, I'd say the hardest part of the project was just getting Git to behave in VS Code. Otherwise, I was a bit worried at first making a webpage with so many others, but having defined jobs really made it easier for me. Originally I had planned to add pictures to the tiles, but that proved to be far much more work than it was worth. Having forgone that, I still tried to design them something similar to Amazon tiles, but a little more compact since space was saved by removing pictures, so you could see more at once. I did learn something I hadn't noticed before: the difference between span and div when making classes in the same heading. Div forces a new line while, span doesn't. Otherwise, remembering how button formatting in CSS worked was a little difficult, but fun.

4.3 Jarrod

4.3.1 JSON File

I was in charge of creating the JSON file and fetching the data from the JSON file. I went with a structure of each product having a unique productID followed by a Product Name, Price, Seller, Category, Description and Availability. These would all be relevant when displaying the products on tiles.

For fetching that data from the JSON file I decided on using a useEffect for pulling in the products. It preserves the raw dataset, keeping the full products list intact. Also ensures that you do not lose any items when filtering, allowing users to clear or change their search without re-fetching from the file. Doing it this way also separates lifecycles for better performance ensuring that the filtering logic only runs when the search text or selected category actually changes, rather than executing on every single component re-render.

4.4 Isaiah

4.4.1 Interface

As the web designer for GenStore, I was responsible for planning the visual architecture, establishing the responsive layout system, building the persistent shell components (Header and Footer), and styling the application for an authentic e-commerce feel.

I began by drafting the Interface Sketch (Figure 2) to translate the project requirements into a concrete UI layout. The architectural plan called for:

1. A sticky top header for persistent site branding and quick access to social media channels.
2. A clean, responsive footer anchored to the bottom of the viewport containing store navigation and legal notices.

To ensure project cleanliness as the codebase expanded, I also refactored our styles by organizing all stylesheets into a centralized `src/Styles/` directory and standardizing their imports across components.

4.4.2 Header Component and Responsive Social Bar

In `Header.jsx` and `Header.css`, I created the top application frame. The header utilizes Flexbox to anchor the store title on the left and house the `SocialBar` component on the right. To give the store a modern, polished look, I configured the header with sticky positioning (`position: sticky; top: 0;`) with a semi-transparent blended background (`color-mix(in srgb, var(--bg) 95%, transparent)`) and a subtle hover transition.

To present the social media channels cleanly, I styled the `.socialBar` using CSS Grid with four auto-sized columns on desktop (`grid-template-columns: repeat(4, auto);`). Violet had initially imported the static social icons at 20px, but they appeared too small in the context of the header banner. I scaled them up to 32px for improved visibility and added hover transitions. Furthermore, to accommodate mobile devices, I added a media query for viewports $\leq$ 640px that collapses the social bar into a 2×2 grid and scales the icons to 1.5rem, preventing any horizontal clipping or layout distortion.

4.4.3 Footer Component and Dynamic Navigation

In `Footer.jsx` and `Footer.css`, I implemented the bottom boundary of the application. Beyond a simple copyright line, I built out an entire navigation links section modeled after production e-commerce platforms, including `Support`, `FAQ`, `Report`, `Become a seller`, and `Orders`.

The footer is styled with Flexbox in a column orientation with consistent spacing (`gap: 1.25rem`). To ensure that the navigation links remain fully responsive across various device form factors, I configured `.footerLinks` with `display: flex`, `justify-content: space-around`, and `flex-wrap: wrap`. On narrower phone screens, the links naturally wrap into tidy rows rather than overflowing the viewport or introducing horizontal scrollba rs. I also added smooth text color transitions on hover (`color: var(--accent)`) and a centered copyright banner.

4.4.4 Global E-Commerce Theming and Palette

In `index.css`, I overhauled the default starter styling to implement an e-commerce theme with high visual contrast and accessibility. I defined semantic CSS root variables for both light and dark modes:

- `--bg` and `--bg-edge`: Provide a rich background with subtle linear gradient borders.
- `--text` and `--text-h`: High-contrast colors for body text and headers.

- `--accent`, `--accent-bg`, and `--accent-border`: Vibrant blue tones used for interactive elements, focus states, and link hovers.
- `--social-bg` and `--border`: Neutral tones for component cards, input fields, and structural separators.

By configuring `@media (prefers-color-scheme: dark)`, the store dynamically respects the user's operating system preferences, switching between a clean, crisp light storefront and an eye-friendly dark shopping environment without requiring JavaScript state.

4.4.5 Form Styling, Container Spacing, and Product Grid Alignment

To bring the entire application together into a unified experience, I polished the interactive elements in `App.css` and `ProductGrid.css`:

- **Container Spacing:** Added horizontal padding (`padding-left: 1rem; padding-right: 1rem; box-sizing: border-box;`) across `.listingSection`, `.filterForm`, and `.products`. This eliminated an issue where input fields and product grids collided directly with the application border on mobile screens.
- **Input Field Polish:** Standardized the select dropdowns, search input, and CRUD input fields with thematic backgrounds (`var(--social-bg)`), rounded borders (`border-radius: 6px`), comfortable padding, and custom glowing focus rings (`box-shadow: 0 0 0 3px var(--accent-bg)`).
- **Action Buttons:** Applied `text-transform: capitalize` to `.listingSubmit`. This allowed the action button text to display in clean title case ("Create", "Update", "Delete") while allowing the underlying JavaScript mode state to remain in lowercase, preventing case-mismatch bugs.
- **Product Grid Centering:** In `ProductGrid.css`, I added a mobile media query for screens $\leq$ 544px with `justify-content: center` and `justify-items: center`, ensuring that when the product grid collapses to a single column, product cards remain centered rather than skewing awkwardly to the left.

5 Testing

Most of the testing for this program was done via the test() function in App.jsx. The test function gets run on mount via a useEffect(), this is to prevent it running multiple times. Many functions had to be redesigned in ways that would allow them to run without the state variables. State variables can be used for testing but they cannot be set using their setState function. The setState function works lazily in that it will only change state on next render. Seeing as the useEffect with an empty dependency array only runs on first mount, when the component is first being rendered these setStates will never work withing the context of the test function. State variables can be used but they must be set directly rather than via the setState function provided by useState(). Typically you should avoid this as the purpose of useState is to create variables who can

trigger rerenders of dependencies but since the test function is based on assertion tests and isn't concerned with rendering this is alright. Additionally the products fetching function had to be created as the test function must be able to pull in the products to run its tests. To trigger testing the TESTING_ENV variable must be set to true. This is not an environment variable unlike its name may suggest, instead it sits in the code directly and must be edited by the programmer to enable or disable it.

The different test cases utilize some shared variables and functions listed below

Listing 1: Persistent variables of test()

```
const fAssert = (value, expected=true, msg="Assertion test failed") => {
    if (value !== expected) console.error("Assertion Fail: " + msg);
}

const prods = await fetchProducts();
let filtered;
```

5.1 Test Cases

This section will go over the various test cases conducted.

5.1.1 Fetching Products

A test case was written to test the fetching of products from the JSON file. This test case required that the previous logic for fetching products be changed. Before the test case the fetching of the products was written once inside a useEffect, this works but it isn't reusable. Since state doesn't change until the next render we can't rely on a useEffect to update the state with a setState like it previously had been. Instead the logic for fetching products was extracted into an async function. This allowed the logic of the useEffect to remain the same but instead of chaining the Promise returned by the fetch it instead just used one then clause to set the states and then catches any errors that might of been tossed by the function. The function now grabs the data from the JSON instead meaning it can be used in multiple places of the code. This allows the test() function to get a hold of the products before first render by calling the function. Using this we pull in the products from the JSON in our test() function and assign it to a variable. Then to run a basic test we check to see if the prods variable has a length greater than 0. This doesn't check that we got all the products or even the right ones but it checks that we got something. Checking if we got the right products would mean hard-coding the right values into the test function which creates tight coupling between products.json and the test() function which is undesirable and harms our ability to scale.

Listing 2: fetchProducts() function

```
/**
* Fetches the products from the JSON file
* @returns The products fetched and formatted
*/
```

```
async function fetchProducts() {
    const response = await fetch("/products.json");

    if (!response.ok) throw new Error("Failed to fetch products");

    const json = await response.json();

    return json.map(prod => ({
        ...prod,
        productCategories: prod.productCategories.map(cat => cat.toLowerCase())
    }));
}
```

Listing 3: Test Case

```
const prods = await fetchProducts();
fAssert(prods.length > 0, true,"Products did not pull in correctly");
```

5.1.2 No Match Checks

Two test cases were written to check if the filtering would properly return no products when filtering for something with no matches. Both checks relied on an obscure character sequence that would be unlikely to be found in any part of the products. Ideally we would prevent these characters from being used but for now this obscurity should work fine. These characters are so obscure they cannot be plain typed into this document as they are not UTF-8 characters, in this document they will be referred to as 'OBSCURECHARACTERS'

The first of two checks was a category select check. By setting the categorySelected to the obscure sequence of characters then running filterProducts we will get the products filtered by that category. A successful test is one in which no products are returned

```
// CHECK FOR NO CATEGORY MATCH
searchText = "";
categorySelected = "OBSCURECHARACTERS";  // Shouldn't be any categories for this
filtered = filterProducts(prods);
fAssert(filtered.length, 0, "Returned products for invalid category filter");
```

The next check was the same idea but it was conducted on the search text instead of the category

```
// CHECK FOR NO SEARCH MATCH
searchText = "OBSCURECHARACTERS";  // Such an obscure character combo no one wou
categorySelected = "all";
filtered = filterProducts(prods);
fAssert(filtered.length, 0, "Returned products for invalid search filter");
```

5.1.3 No Filter Check

A test case was written to see if when no filters were applied the filterProducts() function would return all products. A successful test is one in which the products from the JSON file have the same length as the products after filtering. We cannot check for equality on this as filterProducts() also alphabetizes the products meaning they do not preserve order.

```
// CHECK FOR NO FILTERING
categorySelected = "all";
searchText = "";
filtered = filterProducts(prods);
fAssert(filtered.length, prods.length, "Did not pass check for no-filter");
```

5.1.4 Shallow Sort Validation

A test case was written to test whether filtered products were being alphabetized. This check is shallow and does not check beyond the first character or for string length concerns. This test has been deemed sufficient for most applications and a more in-depth test may be written later if necessary. The code maps each product in filtered to the character code of its first character. A for loop then goes through the character code array and ensures that every character code is greater than or equal to the preceding ones by tracking the previous biggest code. A successful test is one in which no character following another has a smaller character code.

```
// CHECK ALPHABETICAL - LIGHT CHECK AND ONLY CHECKS FIRST LETTER
categorySelected = "all";
searchText = "";
filtered = filterProducts(prods);
filtered = filtered.map(prod => prod.productName.charCodeAt(0));
let biggest = 0;

for (const cc of filtered) {
    fAssert(cc >= biggest, true, "Light check on alphabetic order failed");
    biggest = cc;
}
```

5.1.5 CRUD

Tests were written to assure that products were being created, deleted, and updated properly. This required remodelling the functions to be able to work when the app wasn't running. I just created a fake Event object that had any properties accessed by the functions available. To the function it just looks like a normal object. In the process of this I found some errors in how numbers were being saved to products where they weren't being parsed. I didn't think this was a problem until I realized search filtering by price fails when this isn't a number so I changed those things.

Various tests were done outside the test function where I noticed that the product tiles were not properly handling multiple categories so I fixed that in ProductTile.

5.2 Debugging

5.2.1 Filtering - Violet

Upon writing tests for the filtering many errors were found in the filtering code. The first error encountered was that the category select filter was not being applied. The problem was a simple one, the code wasn't storing the filtered array after category filtering. The Arrays.filter() method does not mutate the original array instead returning a new array with the filter applied. My mistake was not setting filteredProducts equal to this new value. The new code after the fix is listed below

```
if (category !== "all")
filteredProducts = filteredProducts.filter(
    prod => prod.productCategories.includes(category)
);
// Filter by category dropdown, productCategories should already be lowercase
```

Further testing of the filtering code revealed another error in which the filtering was not using the most recent filters instead using the previous ones. This was a case of state not updating. As mentioned in the Testing section, state updates lazily waiting till next rerender to reflect that new state. This can cause issues for states that aren't used by components such as the filters. This could be fixed in many way including adding the filters somewhere in the component to force it to rerender on their change. Instead I just opted to add a useEffect with the filter states as dependencies. The new code had the inputs on the page update the state of the filters which in turn triggered the effect causing it to refilter the products with the most recent data. The updated code is below

```
// Filtering stuff
let [filteredProducts, setFilteredProducts] = useState([]);
let [searchText, setSearchText] = useState("");
let [categorySelected, setCategorySelected] = useState("All");

/**
* Updates the state of the filters and then updates the filtered products trigge
* @param {Event} e The event that spurred this action
*/
function updateFilters(e) {
    switch (e.target.className) {
        case "selectFilter":  // Category selection changed
        setCategorySelected(e.target.value);
        break;
        case "searchFilter":  // Search query changed
        setSearchText(e.target.value);
        break;
```

```
    }
}

useEffect(() => {
    setFilteredProducts(filterProducts);
}, [categorySelected, searchText]);

/**
* Filters the products by the current filtering queries, alphabetizes the results
* @param {Array<Object>} prods Products to use instead of the state products, us
* @returns {Array[Object]} an array of the products filtered and sorted
*/
function filterProducts(prods=null) {
    // FILTERING CODE OMMITTED
    return filteredProducts.sort(
        (a, b) => alphabetCompare(a.productName, b.productName)
    );
}
```

5.2.2 Sorting - Violet

The filtering function performs and alphabetic sorting of the products by their name. This ensures the products are in order of their names ascending. The logic for this however did not work in initial testing. The error was that alphabetCompare, the function that tells Arrays.sort which item comes first, was being passed the product objects instead of the product names. This was a simple fix just needing to change it from passing the parameters a and b to passing a.productName and b.productName. The updated code is below

```
/**
* Compares two strings and returns a number indicating which comes first alphabe
* @param {String} a The first string
* @param {String} b The second string
*/
const alphabetCompare = (a, b) => {
    if (a === b) return 0;      // They are the same string so it doesn't matter

    for (let i = 0; i < a.length && i < b.length; i++) {
        if (a.charCodeAt(i) > b.charCodeAt(i)) return 1;
// b's letter comes first
        else if (a.charCodeAt(i) < b.charCodeAt(i)) return -1;
// a's letter comes first
    }

    // They have the same characters up to some point meaning we assess string l
    return (a.length > b.length) ? 1 : -1
```

```
}

/**
* Filters the products by the current filtering queries, alphabetizes the results
* @param {Array<Object>} prods Products to use instead of the state products, us
* @returns {Array[Object]} an array of the products filtered and sorted
*/
function filterProducts(prods=null) {
    // FILTERING CODE OMITTED
    return filteredProducts.sort(
        (a, b) => alphabetCompare(a.productName, b.productName)
    );
}
```

5.2.3 Footer Viewport Floating - Isaiah

During early development and testing of the empty product state, a layout bug occurred where the footer would float in the middle of the screen whenever there were few or no products displayed. Because the main content did not fill the vertical height of the browser window, the normal document flow placed the footer immediately below the listing form, leaving a large awkward blank space between the footer and the bottom of the screen.

To fix this, I set up a CSS Flexbox sticky-footer architecture. In `index.css`, I styled the root application container (`#root`) to have a minimum height equal to the full viewport height (`min-height: 100svh;`) and configured it as a flex column. Then, in `Footer.css`, I gave the `footer` element `margin-top: auto;`. This instructs the flex layout engine to automatically consume all remaining vertical space above the footer, pushing it securely to the bottom of the viewport when content is short, while allowing it to naturally scroll down when the product catalog is populated.

Listing 4: Sticky Footer Flexbox Implementation

```
/* In src/Styles/index.css */
#root {
  width: 1126px;
  max-width: 100%;
  margin: 0 auto;
  border-inline: 1px solid var(--border);
  min-height: 100svh;
  display: flex;
  flex-direction: column;
  box-sizing: border-box;
  background-color: var(--bg);
}

/* In src/Styles/Footer.css */
```

```
footer {
  border-top: 1px solid var(--border);
  border-bottom: 1px solid var(--border);
  background-color: color-mix(in srgb, var(--bg) 95%, transparent);
  margin-top: auto;
  padding: 1.5rem 2rem 1rem;
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
}
```

5.2.4 Header Social Bar Mobile Collision - Isaiah

When testing responsiveness on mobile viewports ($\leq$ 640px), the header layout broke. Violet's `SocialBar` component originally rendered the four social media icon links in a single horizontal row. On narrow mobile displays, this 4-column layout consumed too much horizontal space, colliding with the GenStore title and causing the header elements to either clip out of the viewport or force an unwanted horizontal scrollbar.

To resolve this issue, I converted the social bar container from a rigid horizontal layout into a responsive CSS Grid in `Header.css`. On desktop viewports, it displays four auto-sized columns. In a media query targeted at screens $\leq$ 640px, the grid reconfigures into two columns (`grid-template-columns: repeat(2, auto);`) with reduced gap spacing and scaled-down icon graphics (1.5rem). This preserved one-tap accessibility for all social channels without distorting the header proportions or spilling over the screen edges.

Listing 5: Responsive Social Bar CSS Grid

```
/* In src/Styles/Header.css */
.socialBar {
    display: grid;
    grid-template-columns: repeat(4, auto);
    gap: 1.5rem;
    align-items: center;
}

.socialTile img {
    width: 32px;
    height: 32px;
}

@media (max-width: 640px) {
    .socialBar {
        grid-template-columns: repeat(2, auto);
        gap: 8px;
```

```
    }

    .socialTile img {
        width: 1.5rem;
        height: 1.5rem;
    }
}
```

5.2.5 Form Edge Collision and Focus Glow Feedback - Isaiah

Another issue discovered during UI testing was that input elements in the filter form and the CRUD listing section lacked consistent boundary padding. On smaller screens, the text inputs and dropdowns touched the outermost border of the root container directly. Furthermore, active input fields lacked clear visual feedback, making it difficult for users to discern which field was currently focused when editing listings.

To resolve these issues, I introduced uniform horizontal padding and border-box sizing across `.listingSection`, `.filterForm`, and `.products` in `App.css`. Additionally, I applied custom focus states using the thematic accent variables, eliminating the default browser outline and adding a smooth glow effect.

Listing 6: Input Container Spacing and Focus Feedback

```
/* In src/Styles/App.css */
.listingSection,
.filterForm,
.products {
  padding-left: 1rem;
  padding-right: 1rem;
  box-sizing: border-box;
}

.selectFilter:focus,
.searchFilter:focus,
.listingSelect:focus,
.productNameInput:focus,
.productPriceInput:focus,
.productSellerInput:focus,
.productCategoriesInput:focus,
.productDescriptionInput:focus {
  outline: none;
  border-color: var(--accent);
  box-shadow: 0 0 0 3px var(--accent-bg);
}
```

6 Figures

Figure 1: Component Heirarchy Diagram

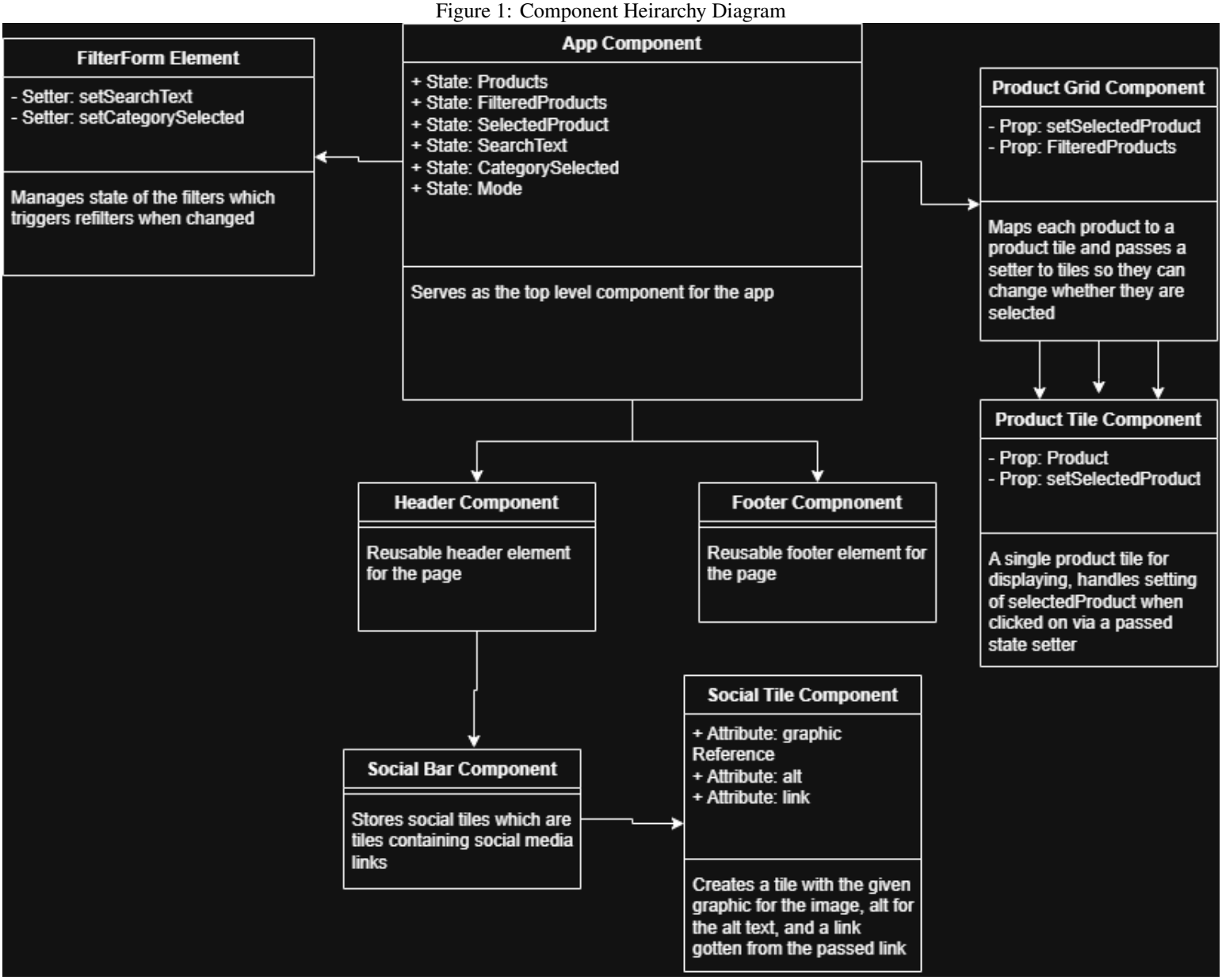


Figure 2: Interface Sketch

